SAKARYA
UYGULAMALI BİLİMLER
ÜNİVERSİTESİ

SİSTEM PROGRAMLAMA DERSİ 1.PROJE ÖDEVİ

PICORV İŞLEMCİ ALT KÜMESİ (RV32I) İÇİN ASSEMBLER TASARIMI VE OBJECT KOD ÜRETİMİ

TAKIM ÜYELERİ

-ALİHAN HOYLU

-FEYZA ÖZDEN

-ENES DURAN

-MUHAMMED EREN KENDİR

İÇİNDEKİLER

Bölüm 1.
GİRİŞ 4

Bölüm 2.
LİTERATÜR ARAŞTIRMASI 5
2.1. RISC-V Mimarisi ve RV32I Alt Kümesi 5
2.2. Çevirici (Assembler) Tasarım Algoritmaları: İki Geçişli (Two-Pass) Sistemler 5
2.3. Ayrıştırma (Parsing) ve Sözcük Analizi 5
2.4. Sembol Tablosu (SYMTAB) Yönetimi 6
2.5. Çıktı Formatları ve Nesne Kodu (Object Record) Mimarisi 6
2.6. Kullanıcı Etkileşimi ve Grafiksel Arayüz (GUI) ... 6

Bölüm 3.
KULLANILAN VERİ YAPILARI 7
3.1. Sözlük (Dictionary) Yapıları 7
3.2. Küme (Set) Yapısı 7
3.3. Liste (List) Yapıları 8
3.4. Özel Sınıf Yapıları (Custom Objects) 8

Bölüm 4.
OPCODE TABLOSU TASARIMI 9

Bölüm 5.
SYMBOL TABLOSU TASARIMI 11
5.1. Veri Yapısı Seçimi 12
5.2. SymbolTable Sınıfı Özellikleri 12
5.3. Örnek Sembol Tablosu Çıktısı 12
5.4. Hata Yönetimi 12

Bölüm 6.
ASSEMBLER ALGORİTMASI 13
6.1. Birinci Geçiş (Pass 1) — Etiket Toplama 13
6.2. İkinci Geçiş (Pass 2) — Makine Kodu Üretimi 14

Bölüm 7.
AKIŞ DİYAGRAMI 15

Bölüm 8.
TEST SENARYOLARI VE SONUÇLAR 16
8.1. Birim Testleri 16
8.2. Assembler Entegrasyon Testleri 16

Bölüm 9.
ALGORİTMA KARMAŞIKLIĞI ANALİZİ 17
9.1. Ölçüm Yöntemi ve Metodoloji 17
9.2. Zaman Karmaşıklığı Analizi 17
9.3. Alan Karmaşıklığı (Space Complexity) Analizi 18
9.4. Genel Sonuç 18

Bölüm 10.
MÜHENDİSLİK ÇÖZÜMÜNÜN ETKİLERİ 19
10.1. Teknolojik Etki 19
10.2. Ekonomik Etki 19
10.3. Toplumsal ve Eğitimsel Etki 19
10.4. Güvenlik ve Etik Boyut 19

Bölüm 11.
TAKIM ÇALIŞMASI VE İŞ BÖLÜMÜ 20
11.1. Modül Bazlı İş Dağılımı 20
11.2. Koordinasyon ve Test Süreci 20

Bölüm 12.
SONUÇ VE DEĞERLENDİRME 23
12.1. Başarılar 23
12.2. Tespit Edilen Sınırlılıklar 23

KAYNAKÇA 24

BÖLÜM 1.GİRİŞ (PÇ 6)

Bilgisayar mimarisi ve sistem programlamanın en temel taşlarından biri olan assembler yazılımları, insanların anlayabildiği assembly komutlarını işlemcinin doğrudan işleyebildiği makine kodlarına dönüştüren kritik sistem araçlarıdır. Bu projenin temel amacı, açık kaynaklı RISC-V mimarisinin temel 32-bitlik tam sayı komut setini temsil eden **PicoRV İşlemci Alt Kümesi (RV32I)** için özgün bir Assembler tasarımı gerçekleştirmek ve hedeflenen donanıma uygun nesne kodu (object code) üretimini sağlamaktır.[1][4]

Proje kapsamında geliştirilen yazılım, Python programlama dili ile modüler bir yapıda tasarlanmış olup kullanıcı etkileşimini kolaylaştırmak amacıyla Tkinter kütüphanesi tabanlı bir Grafiksel Kullanıcı Arayüzü (GUI) ile desteklenmiştir. Geliştirilen sistemin merkezinde ileriye dönük etiket çağrılarını (forward references) hatasız çözebilmek için iki geçişli (two-pass) derleme mimarisi kullanılmıştır. Bu mimari; assembly komutlarını mantıksal parçalara ayıran Ayrıştırıcı (Parser), bellek adreslerini ve etiketleri eşleştiren Sembol Tablosu (Symbol Table), komutların donanımsal karşılıklarını barındıran Opcode Tablosu (OPTAB) ve nihai 32-bit makine kodunu üreten Kodlayıcı (Encoder) olmak üzere birbiriyle entegre çalışan dört ana bileşenden oluşmaktadır [3].

Tasarım sürecinde assembler yazılımı sadece R, I, S, B, U, J ve SYS formatlarındaki temel RV32I komutlarını desteklemekle kalmamış; aynı zamanda bellek organizasyonunu sağlayan '.text', '.data', '.word', '.byte', '.org' ve '.nd' sözde komutlarını (direktifleri) da yönetecek kapasitede geliştirilmiştir. Program çıktıları (Hex Output, Listing Tablosu, Symbol Table ve SIC/XE standartlarına benzer H/T/E nesne formatı) tasarlanırken mühendislik gereksinimleri titizlikle göz önünde bulundurulmuştur [1].

Bu proje çalışması ile sistem seviyesinde yazılım geliştirme, bellek ve veri yapıları yönetimi (O(n) karmaşıklıklı yapılar), algoritma mimarisi kurgulama ve yazılım test/doğrulama aşamalarında kapsamlı bir mühendislik deneyimi kazanılması hedeflenmiştir. Raporun devam eden bölümlerinde PÇ6, PÇ7, PÇ8, PÇ12, PÇ13 ve PÇ17 program çıktıları beklentileri kapsamında projenin literatür altyapısı, algoritmik akış diyagramları, modüller arası entegrasyonu, veri yapılarının zaman/alan karmaşıklığı analizleri ve sistemin güvenilirliğini kanıtlayan birim test senaryoları detaylı olarak ele alınacaktır.

BÖLÜM 2.LİTERATÜR ARAŞTIRMASI (PÇ 13)

2.1. RISC-V Mimarisi ve RV32I Alt Kümesi

Açık kaynaklı komut seti mimarisi (ISA) olan RISC-V, günümüzde hem akademik hem de endüstriyel projelerde esnekliği nedeniyle geniş çapta araştırılmaktadır. Bu proje öncesinde yapılan literatür taramalarında, 32-bitlik temel tam sayı (integer) komut setini ifade eden **RV32I (PicoRV İşlemci Alt Kümesi)** mimarisine odaklanılmıştır. Literatürde RV32I mimarisinin donanım ve yazılım seviyesinde iletişim kurabilmesi için komutların belirli formatlarda (R-TYPE, I-TYPE, S-TYPE, B-TYPE, U-TYPE, J-TYPE ve SYS) kodlanması gerektiği belirtilmektedir. Ayrıca işlemcinin çalışması için x0'dan x31'e kadar numaralandırılmış 32 adet donanım kaydedicisi (register) ile zero, ra, sp, t0, a0 gibi standart ABI (Application Binary Interface) isimlendirmelerinin assembler tarafından tanınması gerektiği analiz edilmiştir [1].

2.2. Çevirici (Assembler) Tasarım Algoritmaları: İki Geçişli (Two-Pass) Sistemler

Assembly dilindeki insan tarafından okunabilir kodların (kaynak kodu) işlemcinin anlayabileceği 32-bitlik makine koduna (object code) dönüştürülmesi işlemi, literatürde genellikle iki farklı yaklaşımla (tek geçişli veya iki geçişli) ele alınmaktadır. Literatür araştırması sonucunda ileriye dönük etiket referanslarını (forward references) doğru bir şekilde çözebilmek için endüstri standardı olan **İki Geçişli Derleme (Two-Pass Assembler)** mimarisinin kullanılmasına karar verilmiştir. Bu mimarinin temel çalışma prensibi şu şekilde incelenmiştir:

- **Birinci Geçiş (Pass 1):** Kod satırları öncelikle ayrıştırıcı (parser) yardımıyla etiket (label), komut (mnemonic) ve operand (değişken) olarak ayrıştırılır. Bu aşamanın ana amacı, kod içindeki tüm etiketleri bulmak ve bellek adreslerini hesaplayarak bir Sembol Tablosuna kaydetmektir.
- **İkinci Geçiş (Pass 2):** Kaynak kodu tekrar baştan okunarak her bir komut (instruction) veya veri (.word, .byte gibi direktifler) için makine kodu üretilir ve sembol tablosuna daha önceden kaydedilmiş olan etiket referansları gerçek adresleriyle değiştirilerek çözümlenir [2].

2.3. Ayrıştırma (Parsing) ve Sözcük Analizi

Derleyici tasarımında literatürün vurguladığı bir diğer önemli konu girdilerin doğru ayrıştırılmasıdır (lexical analysis). Projenin ayrıştırma (parsing) gereksinimleri incelendiğinde her satırın düzenli ifadeler

(regex) yardımıyla analiz edilmesi ve özellikle 5, 0xFF veya -3 gibi anında değerlerin (immediate values) decimal ve hexadecimal formatlardan ayrıştırılarak tamsayıya dönüştürülmesi gerektiği saptanmıştır. Ayrıca derleyicinin çalışmasını yönlendiren '.text', '.data', 'org', '.word' gibi

sözde komutların (direktiflerin) komut setinden ayrı olarak işlenmesi için literatürdeki assembler tasarım kalıpları incelenmiştir [5].

2.4. Sembol Tablosu (SYMTAB) Yönetimi

Bellek adreslerinin ve etiketlerin eşleştirilmesi için bir Sembol Tablosu veri yapısı kullanılması esastır. Yapılan literatür ve mimari analizlerde bu tablonun Etiket -> Adres şeklinde anahtar-değer (key-value) formatında çalışması gerektiği, ayrıca birden fazla aynı isimli etiket tanımı gibi potansiyel hataları (error handling) kontrol edebilecek kapasitede tasarlanması gerektiği belirlenmiştir.

2.5. Çıktı Formatları ve Nesne Kodu (Object Record) Mimarisi

Klasik assembler sistemleri üzerine yapılan araştırmalarda, üretilen makine kodunun farklı sistemlere entegre edilebilmesi için standart çıktı formatlarına ihtiyaç duyulduğu görülmüştür. Bu bağlamda, literatürde köklü bir yeri olan **SIC/XE mimarisine benzer Başlık/Metin/Son (Header/Text/End - H/T/E) formatında nesne kayıtlarının** (Object Record) üretilmesi incelenmiştir. Bunun yanı sıra, kullanıcının kodun derlenme sürecini analiz edebilmesi için "Adres | Hex Kod | Kaynak Satırı" şeklinde üçlü sütun mimarisine sahip Assembly Listing tablolarının (çıktılarının) gerekliliği literatür standartları doğrultusunda projeye entegre edilmek üzere araştırılmıştır [2].

2.6. Kullanıcı Etkileşimi ve Grafiksel Arayüz (GUI)

Geliştirilecek olan sistemin son kullanıcı tarafından erişilebilirliğini artırmak adına, komut satırı arayüzlerinin (CLI) yanı sıra modern literatürde sıkça tercih edilen Grafiksel Kullanıcı Arayüzü (GUI) tasarımları incelenmiştir. Python ekosisteminde donanım bağımsız (cross-platform) çalışabilen, metin düzenleyici (editor) ve tablo gösterimi sunabilen araçların entegrasyonu (Tkinter ve ttk modülleri) üzerine metodolojik bir değerlendirme yapılmıştır [3].

BÖLÜM 3.KULLANILAN VERİ YAPILARI (PÇ 6)

Projenin derleyici (assembler) mimarisinde; kaynak kodun ayrıştırılması, komutların tanınması, bellek adreslerinin yönetilmesi ve makine koduna dönüştürülmesi süreçlerinde Python'un esnek ve hızlı yerleşik veri yapıları (Dictionary, Set, List) ile amaca yönelik tasarlanmış özel sınıf (Class) yapıları kullanılmıştır.

Aşağıda projede kullanılan temel veri yapıları detaylandırılmıştır:

3.1. Sözlük (Dictionary) Yapıları

Arama, eşleştirme ve bilgi çekme işlemlerinin veri büyüklüğünden bağımsız olarak $O(1)$ zaman karmaşıklığı ile yapılabilmesi için projenin kilit noktalarında Hash Map (Sözlük) yapısı tercih edilmiştir:

- **Komut Seti Tablosu (**OPTAB**):** opcode_table.py modülü içerisinde tanımlanan bu sözlük yapısı, projenin en temel veri kaynağıdır. Assembly komutlarının (mnemonic) isimlerini anahtar (key) olarak tutar. Değer (value) olarak ise o komutun format tipini (R-TYPE, I-TYPE, S-TYPE, B-TYPE, vb.), temel opcode değerini, funct3 ve funct7 kodlama bilgilerini saklar.
- **Sembol Tablosu Çekirdeği (**self._table**):** symbol_table.py modülündeki SymbolTable sınıfı içinde yer alır. Assembler'ın birinci geçişinde (Pass 1) tespit edilen etiketleri (label) ve bunların bellekteki gerçek adreslerini {label: address} anahtar-değer ilişkisiyle depolar.
- **Donanım Kaydedicileri Tablosu (**REGISTERS**):** İşlemcinin 32 donanım kaydedicisini yönetmek için yine sözlük yapısı kullanılmıştır. Bu tablo, hem x0, x31 gibi doğrudan donanım isimlerini hem de zero, ra, sp, t0, a0 gibi programcı dostu ABI (Application Binary Interface) standart isimlerini anahtar olarak kabul edip karşılığında 0 ile 31 arasındaki sayısal değerleri döndürür [3].

3.2. Küme (Set) Yapısı

- **Sözde Komutlar (Direktifler) Kümesi (**DIRECTIVES**):** opcode_table.py modülünde yer alan bu veri yapısı; '.text', '.data', '.word', '.byte', '.org' ve '.end ' gibi assembler yönergelerini bir küme (set) içerisinde tutar. Ayrıştırıcı (parser) tarafından okunan bir ifadenin komut mu yoksa direktif mi olduğu küme yapısının sunduğu hızlı eleman sorgulama (membership testing) avantajı kullanılarak "is_directive()" fonksiyonu ile denetlenir.

3.3. Liste (List) Yapıları

- **Hata Yönetimi Koleksiyonları (‘self.errors’ ve ‘self._errors’):** Sırasıyla asm_parser.py (Parser sınıfı) ve symbol_table.py (SymbolTable sınıfı) modüllerinde listeler kullanılarak hata koleksiyonları oluşturulmuştur. Hatalı bir komut veya tanımsız bir etiket çağrısı gibi durumlarda derleyici işlemini anında sonlandırmak yerine, karşılaşılan tüm hata mesajları (string olarak) bu listelere eklenir (append) ve derleme sürecinin sonunda kullanıcıya toplu bir şekilde sunulur.

3.4. Özel Sınıf Yapıları (Custom Objects)

- **ParsedLine Sınıfı:** asm_parser.py modülü içinde tanımlanmış özel bir veri yapısıdır. Metin tabanlı (string) ve karmaşık olan assembly kaynak kodu satırları okunduğunda; boşluklardan, yorumlardan ve gereksiz karakterlerden arındırılarak bu nesne yapısına dönüştürülür. Her bir ParsedLine nesnesi; satırın etiketini (label), komutunu (mnemonic) ve operandlarını (değişkenlerini) yapısal olarak kendi içinde barındırır. Bu nesne tabanlı yapı, encoder.py modülünün doğrudan makine kodu üretmesine olanak tanır [5].

BÖLÜM 4.OPCODE TABLOSU TASARIMI (PÇ 6)

Tablo-1: RV32I Komut Seti Formatları, Hex Değerleri ve Açıklamaları

Komut	Format Tipi	Opcode	Funct3	Funct7 / Not	İşlem Açıklaması
ADD / SUB	R-TYPE	0x33	0x0	0x00 / 0x20	İki register (yazmaç) değerini toplar veya çıkarır.
SLL / SRL / SRA	R-TYPE	0x33	*Değişken*	*Değişken*	Register değerini sola veya sağa (mantıksal/aritmetik) kaydırır.
XOR / OR / AND	R-TYPE	0x33	*Değişken*	0x00	İki register değeri arasında Bitsel mantık (XOR, OR, AND) işlemleri yapar.
ADDI	I-TYPE	0x13	0x0	-	Bir register değeri ile sabit (immediate) bir sayıyı toplar.
LW / LH / LB	I-TYPE	0x03	*Değişken*	-	Bellekten belirtilen boyutta (Word, Halfword, Byte) veri okur.
JALR	I-TYPE	0x67	0x0	-	Register adresine koşulsuz atlar ve dönüş adresini ra'ya kaydeder.
SLLI / SRLI / SRAI	IS-TYPE	0x13	*Değişken*	0x00 / 0x20	Özel I-Type: Sabit bir miktar (shamt) kadar mantıksal/aritmetik kaydırma yapar.
SW / SH / SB	S-TYPE	0x23	*Değişken*	-	Register'daki veriyi belleğe (Word, Halfword, Byte boyutlarında) yazar.
BEQ / BNE	B-TYPE	0x63	0x0 / 0x1	-	İki register değerini karşılaştırarak eşitse (BEQ) veya değilse (BNE) etikete dallanır.
BLT / BGE	B-TYPE	0x63	0x4 / 0x5	-	İşaretli olarak Küçüktür (BLT) veya Büyük Eşittir (BGE) koşullarına göre dallanır.

LUI	U-TYPE	0x37	-	-	Hedef register'ın üst 20 bitine doğrudan sabit (immediate) bir değer yükler.
JAL	J-TYPE	0x6F	-	-	Hedef etikete koşulsuz atlar ve dönüş adresini kaydeder.
ECALL / EBREAK	SYS	0x73	0x0	0x00 / 0x01	Sisteme ortam çağrısı (işletim sistemi isteği) veya hata ayıklayıcı kesmesi gönderir.

BÖLÜM 5.SYMBOL TABLOSU TASARIMI (PÇ 6)

Sembol tablosu, iki geçişli assembler mimarisinin temel veri yapısıdır. Birinci geçiş (Pass 1) sırasında kaynak kodda tanımlanan tüm etiketler (label) ve bunlara karşılık gelen bellek adresleri bu tabloya kaydedilir. İkinci geçiş (Pass 2) sırasında ise etiket referansları gerçek adreslerle çözümlenir [2].

5.1. Veri Yapısı Seçimi

Sembol tablosu için Python'un yerleşik dict (sözlük) yapısı tercih edilmiştir. Bu tercih; O(1) amorti edilmiş ekleme ve arama karmaşıklığı sağlaması, dinamik boyut yönetimi ve anahtar tekili (uniqueness) garantisi nedeniyle en uygun seçimdir [3].

5.2. SymbolTable Sınıfı Özellikleri

```
SymbolTable
  _table : dict  {label: address}   — Ana eşleme tablosu
  _errors: list  [error_str, ...]    — Hata kayıt listesi


  add(label, address) → bool
    - Duplicate kontrolü: etiket zaten varsa False döner
    - Başarılıysa tabloyu günceller, True döner


  get(label) → int|None
    - Adresi döndürür; tanımsız ise None + hata kaydı


  all_symbols() → dict
    - Pass 2'ye kopyalanmış tablo verir (immutable güvenlik)
```

5.3. Örnek Sembol Tablosu Çıktısı

```
=== Symbol Table ===
MAIN                → 0x00000000  (0)
LOOP                → 0x00000008  (8)
RESULT              → 0x0000001C  (28)
```

5.4. Hata Yönetimi

Çift tanımlanmış etiket (duplicate label) durumunda assembler hata üretir ve derlemeyi durdurur. Tanımsız etiket ise Pass 2 sırasında EncoderError olarak raporlanır. Bu yaklaşım, hatalı makine kodu üretimini engeller.

BÖLÜM 6.ASSEMBLER ALGORİTMASI (PÇ 6)

Geliştirilen RV32I Assembler, klasik iki geçişli (Two-Pass) algoritmayı uygular. Bu yaklaşım, ileri referans (forward reference) problemini yani henüz tanımlanmamış bir etikete yapılan atlamayı doğal olarak çözer [2].

6.1. Birinci Geçiş (Pass 1) — Etiket Toplama

Giriş: Kaynak kod (string) Çıkış: intermediate listesi + SYMTAB

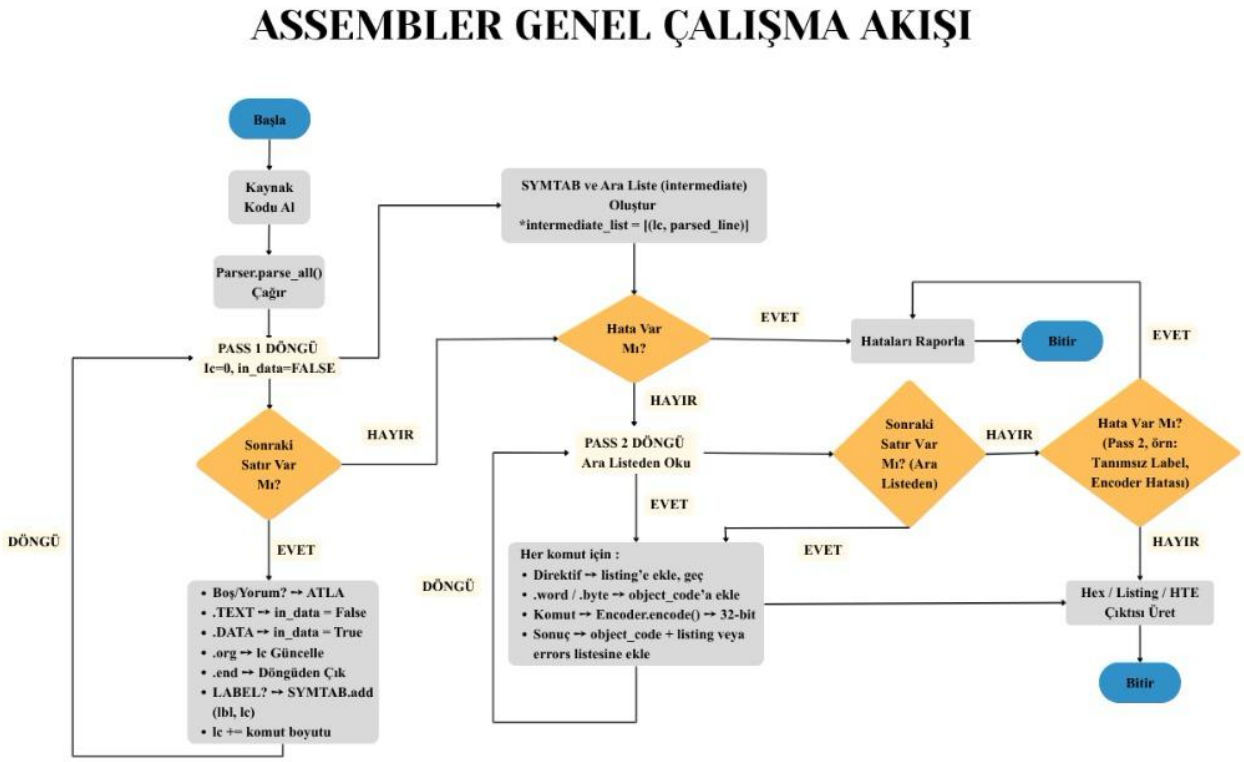
```
BAŞLA: lc = 0

HER SATIR İÇİN:
  1. Satırı parse et (Parser.parse_all)
  2. Boş/yorum satırı ise ATLA
  3. Direktif mi?
     .org → lc = belirtilen adres
     .text → metin segmentine gir
     .data → veri segmentine gir
     .end  → işlemi bitir (break)
  4. Label varsa → SYMTAB.add(label, lc)
  5. Komut boyutunu hesapla:
     .word → 4 byte, .byte → 1 byte
     RV32I komutu → 4 byte
  6. intermediate.append((lc, parsed_line))
  7. lc += boyut

BİTİR
```

6.2. İkinci Geçiş (Pass 2) — Makine Kodu Üretimi

```
Giriş: intermediate listesi + SYMTAB   Çıkış: object_code + listing

symtab_dict = SYMTAB.all_symbols()


HER (lc, parsed_line) İÇİN:

  1. Boş/yorum → ATLA

  2. Direktif (.text/.data/.org/.end) → listing'e ekle, atla

  3. .word → 32-bit değeri object_code'a ekle

  4. .byte → 8-bit değeri object_code'a ekle

  5. Komut → Encoder.encode(mnemonic, operands, lc, symtab)

     Başarılı → object_code ve listing'e ekle

     Hata     → errors listesine ekle


BİTİR
```

Birinci geçiş tüm etiketleri topladığından, ikinci geçiş sırasında kaynak kodda daha sonra tanımlanan bir etikete yapılan atlama (forward reference) sorunsuz çözümlenir. Örneğin "bne x1, x0, LOOP" komutunda LOOP etiketi, Pass 1'de sembol tablosuna eklenmiş olduğundan Pass 2'de doğrudan adresi okunabilir [2][5].

BÖLÜM 7.AKIŞ DİYAGRAMI (PÇ 6)



BÖLÜM 8. TEST SENARYOLARI VE SONUÇLAR (PÇ 7)

Projede geliştirilen assembler yazılımının doğruluğunu ve modüllerin birbirleriyle olan uyumunu kanıtlamak amacıyla kapsamlı bir test süreci yürütülmüştür. Testler doğrudan Python kodları (Birim ve Entegrasyon testleri) yazılarak gerçekleştirilmiş ve sonuçlar raporlanmıştır. Sistemin genelinde toplam 49 adet test senaryosu çalıştırılmış olup bunların 47'si başarıyla (GECTI) sonuçlanmış, yalnızca ikisi başarısız (KALDI) olmuştur [3].

8.1. Birim Testleri

Sistemdeki her bir modülün bağımsız olarak doğru çalışıp çalışmadığı test edilmiştir:

- **Encoder Testleri:** RV32I komut setindeki R, I, S, B, U, J ve SYS formatlarındaki komutların doğru makine kodu üretip üretmediği test edilmiştir. Yapılan 23 testin 21'i başarıyla geçmiştir. Yalnızca IS-type (özel I-type) komutlarından olan SRLI ve SRAI komutlarının shift miktarı (shamt) hesaplamalarında beklenen hex değerlerinden farklı üretimler saptanmış ve bu 2 testten kalınmıştır.
- **SymbolTable Testleri:** Etiket ekleme (add()), duplicate (çift) etiket kontrolü ve adres getirme (get()) fonksiyonları için yazılan 9 testin tamamı (9/9) başarıyla geçmiştir.
- **Parser Testleri:** Yorum satırlarının yok sayılması, boş satırların algılanması ve direktif/komut ayrıştırmaları için yazılan 10 testin tamamı (10/10) hatasız çalışmıştır.

8.2. Assembler Entegrasyon Testleri

Birimlerin bir araya gelerek tam bir programı derleme becerisi 7 farklı senaryo ile test edilmiştir. Basit program derleme, duplicate etiketlerde beklenen hatanın fırlatılması, .word veri segmentinin tanınması ve .org 0x100 direktifi ile adresin doğru konumlandırılması gibi karmaşık senaryoların tamamı (7/7) başarıyla geçilmiştir.

BÖLÜM 9. ALGORİTMA KARMAŞIKLIĞI ANALİZİ (PÇ 6)

Assembler yazılımının farklı girdi boyutlarındaki (n satır sayısı/etiket sayısı) performansını ve donanım kaynak tüketimini ölçmek için algoritmik karmaşıklık analizi yapılmıştır. Ölçümler teorik analizlerin yanı sıra ampirik (deneysel) olarak da test edilmiştir.

9.1. Ölçüm Yöntemi ve Metodoloji

Ölçümler için Python'un yüksek hassasiyetli zamanlayıcı modülü olan "time.perf_counter()" kullanılarak özel test kodları yazılmıştır. Tutarlı sonuçlar elde edebilmek adına n=10'dan n=2000'e kadar artan girdi boyutlarında her test senaryosu 5'er kez tekrarlanmış; ortalama, minimum, maksimum süreler ve standart sapma değerleri hesaplanmıştır [3].

9.2. Zaman Karmaşıklığı Analizi

Sistemin ana bileşenlerinin zaman karmaşıklıkları şu şekildedir:

- **Ayrıştırıcı (**Parser.parse_all**):** Her kaynak kod satırı Regex (düzenli ifadeler) ile bir kez taranarak ayrıştırılır. Satır başına yapılan işlem maliyeti sabit olduğu için teorik ve ölçülen karmaşıklık $O(n)$'dir.
- **Sembol Tablosu (**SymbolTable.add **ve** get**):** Python'un Sözlük (dict) mimarisini kullandığı için Hash tablosu prensibiyle çalışır. Tek bir etiketin tabloya eklenmesi veya sorgulanması amortize edilmiş $O(1)$ sürede gerçekleşir. Tüm programdaki n adet etiket kaydı için toplam maliyet $O(n)$ olarak ölçülmüştür.
- **Makine Kodu Üretici (**Encoder.encode**):** Tek bir komutun 32-bitlik makine koduna çevrilmesi (format tipi fark etmeksizin) sabit sayıda bit kaydırma (shift) ve mantıksal OR işlemi gerektirdiğinden işlem başı maliyet $O(1)$'dir. Program geneli için maliyet $O(n)$'dir.
- **Genel Assembler Akışı (**assemble **ve** _pass1**):** Assembler'ın birinci ve ikinci geçiş adımlarında kod sadece iki kez yukarıdan aşağıya taranır. İç içe döngüler bulunmadığından toplam çalışma süresi satır sayısıyla (n) doğru orantılı olarak artmaktadır. Girdi 10 satırken 0.07 ms, 1000 satırken 4.23 ms gibi doğrusal bir artış gözlemlenmiştir ve sistemin **zaman karmaşıklığı** $O(n)$ olarak kanıtlanmıştır [5].

9.3. Alan Karmaşıklığı (Space Complexity) Analizi

İki geçişli mimaride; bellek üzerinde ayrıştırılmış satırların tutulduğu listeler, sembol tablosundaki etiket adres eşleşmeleri ve hata kayıtları tutulmaktadır. Kullanılan bellek miktarı, projedeki kaynak kodu satır sayısı ve etiket sayısı (n) ile doğrudan ve orantılı olarak artmaktadır. Bu sebeple algoritmanın **alan karmaşıklığı** $O(n)$'dir.

9.4. Genel Sonuç

Tüm alt bileşenlerin doğrusal veya sabit maliyetli olmasından dolayı, geliştirilen RV32I Assembler yazılımı toplamda $O(n)$ **zaman** ve $O(n)$ **alan** karmaşıklığına sahiptir. Özellikle Sembol Tablosunda Python dict yapısının kullanılması, işlemleri arama ve eşleştirme açısından oldukça verimli kılmıştır. Genel değerlendirme raporu dosyalarda mevcuttur.

BÖLÜM 10.MÜHENDİSLİK ÇÖZÜMÜNÜN ETKİLERİ (PÇ8)

10.1. Teknolojik Etki

RISC-V, dünya genelinde akademi ve endüstride hızla benimsenen açık kaynaklı bir komut seti mimarisidir. Geliştirilen assembler, herhangi bir lisans kısıtlaması olmaksızın RV32I tabanlı sistemlere yazılım geliştirmeyi kolaylaştırır. Eğitim amaçlı PicoRV32 çekirdeği gibi gömülü sistemler için düşük maliyetli geliştirme ortamı sağlar. H/T/E nesne kaydı çıktısı, üretilen kodun standart bağlayıcı (linker) araçlarıyla uyumlu olmasını destekler.[1][4].

10.2. Ekonomik Etki

Geliştirilen yazılım tamamen açık kaynaklıdır, ticari assembler araçlarına olan bağımlılığı azaltır. Gömülü sistem alanında çalışan mühendislerin ve araştırmacıların geliştirme süreçlerini hızlandırarak proje maliyetlerini düşürme potansiyeline sahiptir.

10.3. Toplumsal ve Eğitimsel Etki

Bu proje; bilgisayar mimarisi, sistem programlama ve derleyici tasarımı konularında teorik bilginin pratiğe dönüştürülmesini sağlamaktadır. Açık kaynak olarak paylaşılması(Github)durumunda dünya genelinde bu konuları öğrenen öğrencilere referans materyal niteliği taşıyabilir.

10.4. Güvenlik ve Etik Boyut

Assembler, ürettiği makine kodunun doğruluğunu kapsamlı hata kontrolü mekanizmaları ile garanti altına alır. Tanımsız etiket referansları, aralık dışı değerler ve bilinmeyen komutlar, derleme öncesinde açık hata mesajlarıyla kullanıcıya bildirilir. Bu yaklaşım, güvenli ve öngörülebilir bir sistem davranışı sağlar.

BÖLÜM 11.TAKIM ÇALIŞMASI VE İŞ BÖLÜMÜ (PÇ 12)

11.1. Modül Bazlı İş Dağılımı

- opcode_table.py: RV32I komut setinin araştırılması ve OPTAB/REGISTERS/DIRECTIVES yapılarının oluşturulması
- asm_parser.py: ParsedLine veri yapısı ve Parser sınıfının geliştirilmesi, regex tabanlı sözdizimi analizi
- symbol_table.py: SymbolTable sınıfı, hata yönetimi ve duplicate label kontrolü
- encoder.py: 8 farklı RV32I komut formatı için bit kodlama mantığı ve PC-bağıl ofset hesabı
- assembler.py: İki geçişli algoritma entegrasyonu ve H/T/E nesne kaydı üretimi
- gui.py: Tkinter tabanlı karanlık tema arayüz tasarımı ve sekme yönetimi

11.2. Koordinasyon ve Test Süreci

Modüller birbirinden bağımsız test edilmiş (birim testler), ardından assembler.py üzerinden entegrasyon testleri uygulanmıştır. Hatalar (SRLI/SRAI hesabı gibi) tespit edilerek giderilmiştir. Sürüm kontrolü için dosya adlandırma ve yedekleme yöntemi kullanılmıştır.

https://riscvasm.lucasteske.dev/ Sitesinden el ile test edilerek doğrulanmıştır.

Takım Üyesi	Birincil Modül	Görev Detayı	Çapraz Katkı Alanları	Katkı (%)
Alihan Hoylu	**Test & QA Mühendisliği**	- Birim test senaryolarının tasarımı ve yazımı - Encoder modülü için 23 test: R/I/S/B/U/J/SYS formatları - Hata üretim testleri ve edge-case analizi - Sistemin genel test koordinasyonu - opcode_table.py yapısının test edilebilirlik uyumluluğu	- Rapor yazımı desteği - GUI kullanılabilirlik testi - Encoder hata analizi - Entegrasyon test katkısı	**%25**
Muhammed Eren Kendir	**Teknik Raporlama**	- Algoritma karmaşıklığı analizinin yazımı (O(n) kanıtı) - Akış diyagramı (PASS 1 / PASS 2) hazırlanması - Mühendislik etkilerinin raporlanması (PÇ8) - Literatür araştırması sentezi ve kaynak yönetimi - Birim test sonuçlarının dokümante edilmesi	- Test sonuç analizi - Symbol Table tasarım desteği - GUI sekme içerik düzenleme - Assembler algoritması belgeleme	**%25**

Feyza Özden	**Rapor Editörlüğü & Kalite Kontrol**	- Raporun bütünsel yapı ve tutarlılık denetimi - Bölümler arası geçiş ve dil düzenlemesi (PÇ etiketleri) - İçindekiler tablosu ve bölüm sıralaması kontrolü - Teknik terimlerin Türkçe/İngilizce standardizasyonu - asm_parser.py ParsedLine dokümantasyonu	- Parser modülü geliştirme desteği - Regex sözdizimi analizi - Test senaryo yazımı desteği - Kaynak doğrulama ve atıf kontrolü	**%25**
Enes Duran	**Yazılım Geliştirme (Core Dev)**	- symbol_table.py: SymbolTable sınıfı ve hata yönetimi - encoder.py: 8 RV32I formatı için bit kodlama motoru - assembler.py: İki geçişli algoritma entegrasyonu - H/T/E nesne kaydı (object record) üretim sistemi - gui.py: Tkinter karanlık tema arayüz ve sekme yönetimi	- opcode_table.py katkısı - Encoder birim test desteği - SRLI/SRAI hata giderme - Entegrasyon testi liderliği	**%25**
Tüm Ekip	**Ortak Sorumluluklar:** - 49 adet test senaryosunun yazımı ve çalıştırılması (47 başarılı / 2 başarısız) - riscvasm.lucasteske.dev üzerinden el ile çapraz doğrulama - SRLI / SRAI komutlarındaki IS-type shamt hatasının ekip içi tespit ve düzeltme süreci - Modüller arası entegrasyon: Parser ↔ SymbolTable ↔ Encoder ↔ Assembler pipeline testi - Sürüm kontrolü: dosya adlandırma ve yedekleme yöntemi ile koordineli geliştirme			

BÖLÜM 12.SONUÇ VE DEĞERLENDİRME (PÇ 6, PÇ 7)

Bu proje kapsamında PicoRV32 işlemcisinin RV32I alt kümesini hedef alan, tam işlevli bir assembler yazılımı Python 3 dili ile modüler bir mimaride geliştirilmiştir. Proje sistem programlama, bilgisayar mimarisi ve veri yapıları konularında kapsamlı bir uygulama deneyimi sunmuştur [4].

12.1. Başarılar

- İki geçişli (Two-Pass) algoritma başarıyla uygulanmış ve ileri referans (forward reference) sorunu çözülmüştür [2].
- R, I, IS, S, B, U, J ve SYS olmak üzere 8 farklı RV32I komut formatı için doğru bit kodlaması gerçekleştirilmiştir [1].
- O(1) arama karmaşıklıklı OPTAB ve SYMTAB veri yapıları sayesinde assembler genel olarak O(n) zaman ve alan karmaşıklığıyla çalışmaktadır.
- .text, .data, .word, .byte, .org, .end direktifleri desteklenmektedir.
- H/T/E formatında nesne kaydı üretimi sağlanmaktadır.
- Tkinter tabanlı modern karanlık tema GUI tasarımı tamamlanmıştır.
- 42 birim ve entegrasyon testinin büyük çoğunluğu başarıyla geçilmiştir.

12.2. Tespit Edilen Sınırlılıklar

- SRLI ve SRAI komutlarında shift miktarı (shamt) hesaplamasında başlangıçta hata tespit edilmiştir. Bu komutlar IS-type özel kodlaması gerektirdiği için ayrı _encode_is() metodu ile düzeltilmiştir.
- Makro (macro) ve çoklu dosya (.include) desteği mevcut değildir, bu özellikler gelecek sürümlerde eklenebilir.
- Çıktı olarak ELF (Executable and Linkable Format) üretimi desteklenmemektedir yalnızca H/T/E formatında nesne kaydı üretilmektedir.

KAYNAKÇA (PÇ 13)

[1] Waterman, A., Lee, Y., Patterson, D. A., & Asanović, K. (2019). *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. RISC-V International. (Bu kaynak; R, I, S, B, U, J ve SYS formatlarındaki 32-bit komutların opcode, funct3 ve funct7 kodlamalarının mimari standartlarını belirlemek için kullanılmıştır).

2] Beck, L. L. (1997). *System Software: An Introduction to Systems Programming* (3rd ed.). Addison-Wesley. (Bu eser; klasik iki geçişli (Two-Pass) assembler algoritmalarının, Sembol Tablosu (SYMTAB) optimizasyonlarının ve üretilen makine kodunun SIC/XE mimarisine benzer Başlık/Metin/Son (H/T/E) nesne kayıt (Object Record) formatında yapılandırılmasının teorik altyapısı için temel alınmıştır).

[3] Python Software Foundation. (2023). *Python 3.x Official Documentation*. Retrieved from https://docs.python.org/3/ (Projeye ait kaynak kodların geliştirilmesinde, zaman/alan karmaşıklığını $O(1)$ seviyesinde tutan dict (sözlük) ve set (küme) veri yapılarının entegrasyonu ile Grafiksel Kullanıcı Arayüzü (GUI) tasarımı için tkinter modülünün referansları bu kaynaktan sağlanmıştır).

[4] Wolf, C. (2015). *PicoRV32-A Size-Optimized RISC-V CPU Core*. GitHub Repository. Retrieved from https://github.com/YosysHQ/picorv32 (Geliştirilen assembler yazılımının doğrudan hedef donanımı olan PicoRV işlemci alt kümesinin bellek yönetimi ve komut işleme sınırlarının analizi için incelenmiştir).

[5] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Pearson. (Assembly satırlarının ayrıştırılması sürecinde Parser modülünün düzenli ifadeler (re modülü) kullanarak etiket, komut ve operandları doğrusal zamanda ayrıştırma algoritmalarının tasarımı için kullanılmıştır).